Shadow PC Trial — Complete Setup & Run Guide

From a blank Shadow.tech Windows PC to committed benchmark data

Repository: `github.com/ericmedlock/MSAI-LLM-PERFORMANCE` Guide version: 2026-07-10

ITCS 6881 — LLM Execution Architecture Benchmark

What this guide does

By the end you will have: (1) a Shadow PC with all tooling installed, (2) a completed *frontier-v2 trial* — 36 frozen benchmark tasks × 3 architectures × 1 trial = **108 benchmark runs** against the pinned DeepSeek-R1 14B model on the RTX A4500 — and (3) the results committed and pushed back to GitHub. This is the harness's **first-ever NVIDIA/CUDA environment**, so the trial's real purpose is validating GPU telemetry before the university HPC sweep. Follow the steps in order; each has a verification so you always know whether it worked.

Budget: ∼15 minutes of installs, ∼10 GB of downloads, then 2–3 hours of unattended running. **Disk needed:** ∼15 GB free.

Conventions used in this guide

- Windows has two different command windows, and it matters which one you use:
  - **PowerShell** — the blue Windows shell. Open it: press the **Start** key, type `powershell`, press Enter.
  - **Git Bash** — installed in Part 2; a Linux-style shell. Open it: Start key, type `git bash`, Enter.

  Every command block below is labeled [**PowerShell**] or [**Git Bash**].
- Type commands exactly as printed, then press Enter. Commands are case-sensitive in Git Bash.
- ☐ marks a verification checkpoint — confirm it before moving on.
- If Windows pops a dialog asking "Do you want to allow this app to make changes?" (a UAC prompt), click **Yes**. This happens during installs; it is expected.

Part 0 — Before you start (Shadow-specific)

1. Connect to your Shadow PC with the Shadow client and stay connected. **Important:** Shadow *shuts the virtual machine down a few minutes after you disconnect.* If that happens mid-run, no data is lost (every finished run is saved instantly), but the benchmark stops until you reconnect and resume. Plan to keep the session open for the full 2–3 hours, or check in periodically.

2. Check free disk space: open **File Explorer** → This PC → the C: drive should show at least 15 GB free.
3. ☐ You are on the Shadow desktop with a working keyboard/mouse and 15+ GB free.

Part 1 — Verify the GPU (30 seconds)

Everything depends on the NVIDIA driver being present. Shadow pre-installs it; verify rather than assume.

[**PowerShell**]

```
nvidia-smi
```

☐ You see a table whose header line includes `NVIDIA RTX A4500` and a memory column reading about `24564MiB` total.

If instead you see "`nvidia-smi is not recognized`": the driver is missing or PATH is stale. Reboot the Shadow PC first; if it persists, update the GPU driver through Shadow's own updater or GeForce/NVIDIA Experience, then repeat this check. **Do not continue until this works** — without the driver, the benchmark will run on CPU (uselessly slow) and the telemetry this trial exists to validate will be empty.

Part 2 — Install Git for Windows (2 minutes)

Git fetches the repository; the bundled *Git Bash* shell is also what the project's scripts (and Claude Code) run in.

[**PowerShell**]

```
winget install --id Git.Git -e --source winget
```

Notes: `winget` is Windows' built-in package manager. If it reports "`winget is not recognized`", install "App Installer" from the Microsoft Store, close and reopen PowerShell, and retry.

Close PowerShell and open a new one (installs only appear in fresh windows).

☐ `git --version` in the new PowerShell prints something like `git version 2.5x`.

Part 3 — Install Python 3.13 (2 minutes)

[**PowerShell**]

```
winget install --id Python.Python.3.13 -e --source winget
```

Again close PowerShell and open a fresh one.

☐ `python --version` prints `Python 3.13.x`. (If it opens the Microsoft Store instead: Start → "Manage app execution aliases" → turn **off** both `python.exe` and `python3.exe` aliases, then retry in a fresh window.)

Part 4 — Install Claude Code (3 minutes)

Claude Code is the AI command-line agent that will drive the whole trial for you (Part 6, Path A). If you prefer to run everything by hand, you may skip this part and use Path B — but Path A also handles surprises for you.

Prerequisite: an Anthropic account with Claude Code access — either a Claude **Pro/Max subscription** or a Console account with API billing. Have those credentials ready before starting; the install below ends in a browser login.

[**PowerShell**]

```
irm https://claude.ai/install.ps1 | iex
```

Open a fresh PowerShell, run `claude`, and follow the login prompts in the browser window it opens (log in with the Anthropic account). Exit Claude Code afterwards by typing `/exit`.

☐ `claude --version` prints a version number without error.

Part 5 — Get the repository (1 minute)

The repository is public — no login needed to download it.

[**PowerShell**]

```
cd ~
git clone https://github.com/ericmedlock/MSAI-LLM-PERFORMANCE.git
```

(`~` means your home folder, e.g. `C:\Users\you`.)

☐ A folder `MSAI-LLM-PERFORMANCE` now exists in your home folder and contains `scripts`, `tasks`, `config`, `results`, `docs`.

Part 6 — Run the trial

Path A — let Claude Code drive (recommended)

Open **Git Bash** (Start key → type `git bash` → Enter), then:

```
cd ~/MSAI-LLM-PERFORMANCE
claude
```

The first time Claude Code opens in this folder it asks "**Do you trust the files in this folder?**" — answer **Yes** (you just cloned this repository yourself from GitHub).

When Claude Code is ready, type:

```
RUN SHADOW PC TRIALS
```

Exact wording is not required — natural phrasings like "get this machine ready and run the shadow eval" trigger the same built-in instructions. The repository ships a skill that tells Claude precisely what to do: run the setup script, verify the GPU telemetry in the first rows, run the trial, then summarize and commit the results. Approve the permission prompts it shows (they correspond to the exact commands in Path B below). You can ask it questions at any time ("how far along is it?", "show accuracy so far").

Path B — manual (identical result, two commands)

Open **Git Bash**:

```
cd ~/MSAI-LLM-PERFORMANCE
bash scripts/setup.sh
bash scripts/run_trials.sh
```

What the setup step does (either path), and what you'll see

1. Detects the platform and prints `[setup] detected profile: shadow`.
2. Creates a private Python environment and installs the pinned dependencies (∼2 min).
3. Runs the offline test suite — ☐ expect `112 passed, 2 deselected`.
4. Installs **Ollama** (the local model server) via winget — expect a UAC prompt — and starts it.
5. Downloads the pinned model `deepseek-r1:14b` (∼9 GB; Shadow's datacenter connection usually does this in a few minutes; a progress bar is shown).
6. Prints the GPU line — ☐ expect `NVIDIA RTX A4500, 24564 MiB` and finally `[setup] DONE`.

The run step then prints a dry-run plan — ☐ expect `Total cells : 108 runs`, writing to `results/frontier-v2-shadow-trial-14b.jsonl` — and starts executing. One line of progress appears per finished run.

Part 7 — The five-minute telemetry check (the point of the trial)

After the first 2–3 runs finish (∼5 minutes in), open a **second** Git Bash window and inspect the first recorded row:

```
cd ~/MSAI-LLM-PERFORMANCE
head -n 1 results/frontier-v2-shadow-trial-14b.jsonl \
  | ./.venv/Scripts/python -m json.tool | grep -iE "vram|gpu|watt"
```

(The \ at the end of the first line lets you press Enter and continue the command on the next line.)

☐ The GPU fields (`peak_vram_mb`, GPU utilization, power) contain **non-zero numbers**. This proves the CUDA/NVML telemetry path works — the single most important output of this trial.

If the fields are zero/empty: stop the run (`Ctrl-C` in the first window — perfectly safe) and investigate before spending hours: is `nvidia-smi` still working? Did Ollama load the model onto the GPU (`ollama ps` should show the model with `100% GPU`)? Report what you find; nothing collected so far is wasted.

Part 8 — During the run

- **Duration:** roughly 2–3 hours for all 108 runs. Math items (AIME) are the slowest — several minutes each is normal; some will legitimately fail (the tier is calibrated to ∼50% single-pass difficulty). Failures are data, not problems.
- **Interrupting is always safe.** Every finished run is written to disk immediately. `Ctrl-C` stops it; re-running `bash scripts/run_trials.sh` continues exactly where it left off.

- **Do not disconnect the Shadow client** (Part 0). If you must, reconnect later and simply re-run the command — it resumes.
- Progress check at any time (second Git Bash window):

  ```
  wc -l results/frontier-v2-shadow-trial-14b.jsonl
  ```

  The number printed is finished runs out of 108.

Part 9 — Send the data home

Pushing requires GitHub authentication once. The easiest route is the GitHub CLI:

[**PowerShell**]

```
winget install --id GitHub.cli -e --source winget
```

Then in a fresh **Git Bash**:

```
gh auth login
```

(choose `GitHub.com` → `HTTPS` → login via browser), and identify yourself to git (once):

```
git config --global user.name "Eric Medlock"
git config --global user.email "ericmedlock@gmail.com"
```

Finally commit and push the results (Path A: Claude Code offers to do this for you):

```
cd ~/MSAI-LLM-PERFORMANCE
git add results/frontier-v2-shadow-trial-14b.jsonl
git add results/host/shadow.json results/hosts.csv
git commit -m "data(shadow):  frontier-v2 trial N=1, RTX A4500"
git push
```

☐ `git push` ends with no error, and on the repository's GitHub page the new file appears under `results/`.

Part 10 — Troubleshooting quick reference

Symptom	Fix
`winget` not recognized	Install "App Installer" from Microsoft Store; fresh PowerShell.
`git`/`python`/`claude` not recognized right after install	Close the window; open a fresh one (PATH updates only apply to new windows).
`python` opens the Microsoft Store	Start → "Manage app execution aliases" → disable the `python.exe` aliases.
`ollama` not recognized after setup	Fresh Git Bash window; or run `bash scripts/setup.sh` again (it is safe to repeat).
Model download very slow / stalled	`Ctrl-C` and re-run setup; Ollama resumes partial downloads.
GPU fields empty in Part 7	See Part 7; check `ollama ps` shows `100% GPU`; check `nvidia-smi`.
Run seems frozen on one item	AIME math items can take several minutes of silent "thinking"; the harness also has its own timeout. Give it 10 minutes before worrying.
Shadow disconnected overnight	Reconnect, open Git Bash, `cd` to the repo, re-run `bash scripts/run_trials.sh` — it resumes.

Final checklist

- ☐ `nvidia-smi` shows the RTX A4500 (Part 1)
- ☐ Git, Python 3.13, Claude Code installed (Parts 2–4)
- ☐ Repo cloned (Part 5)
- ☐ Setup finished: 112 tests passed, model pulled, `[setup] DONE` (Part 6)
- ☐ First-row GPU telemetry is non-zero (Part 7)
- ☐ 108/108 rows in the results file (Part 8)
- ☐ Results committed and pushed to GitHub (Part 9)

Rules the scripts already enforce, stated for completeness: never edit `config/config.yaml`, anything in `tasks/`, or `prompts/` — these are frozen by pre-registration, and the config file's bytes are hashed into every result row. Machine-specific overrides belong in a `.env` file only.